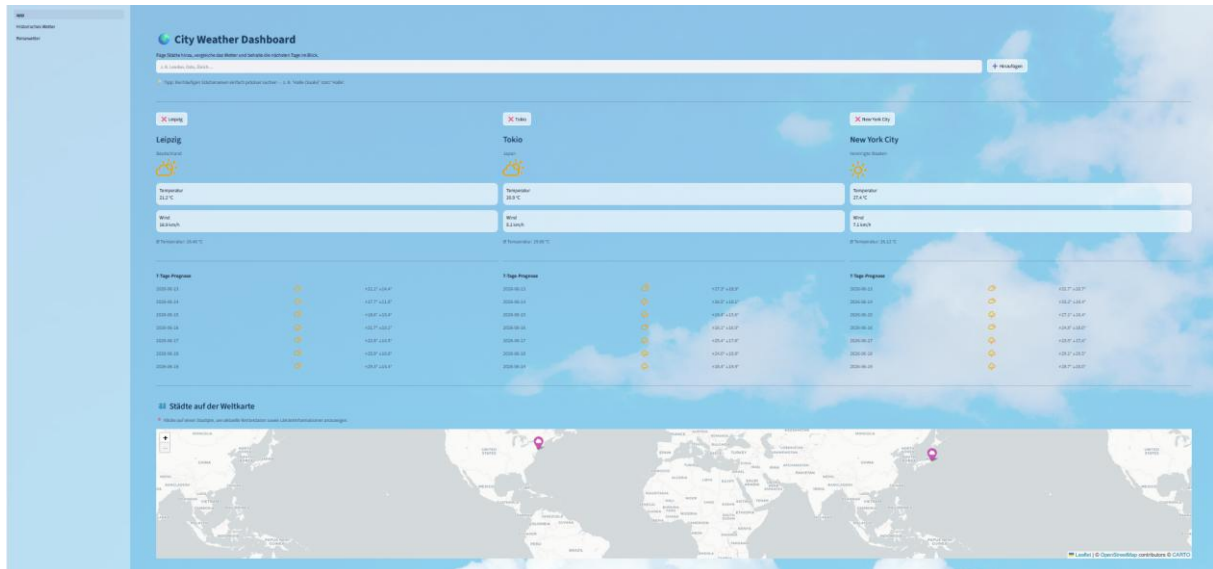


Projekt Dokumentation: „City Weather Dashboard“



Autor: Anne-Katrin Dittmann

Datum: Juni 2026

Version: 1.0

Inhaltsverzeichnis

1. Projektübersicht.....	2
2. Architektur	2
3. Technologiestack	3
4. API-Anbindung	4
4.1 Open-Meteo Forecast API	4
4.2 Open-Meteo Geocoding API	4
4.3 wttr.in (Fallback-API)	5
4.4 REST Countries	5
4.5 OpenStreetMap / CartoDB Positron (Kartendienst)	5
5. Datenbankmodell.....	6
6. Designentscheidungen	8
7. Bekannte Einschränkungen	9
8. Potenziale	9

1. Projektübersicht

Das City Weather Dashboard ist eine webbasierte Datenanwendung zur Verwaltung und dem Vergleich von Wetterdaten beliebiger Städte weltweit. Die Anwendung ermöglicht das Abrufen aktueller Wetterdaten sowie 7-Tage-Prognosen, speichert diese persistent in einer lokalen SQLite-Datenbank und wertet sie statistisch aus. Zusätzlich werden Länderinformationen zu den gespeicherten Städten angezeigt und alle Städte auf einer interaktiven Weltkarte visualisiert.

Über die Navigation sind zwei weitere Seiten erreichbar: Historisches Wetter ermöglicht die Analyse vergangener Wetterdaten beliebiger Orte ab dem Jahr 1940. Reisewetter liefert eine klimatologische Jahresübersicht für beliebige Reiseziele auf Basis von Durchschnittswerten der Jahre 1991–2020.

Bei Ausfall von Open-Meteo springt automatisch wttr.in als Fallback-API ein. Die Auswertung zeigt die sonnigste, verregnetste, wärmste und kälteste Stadt im Vergleich.

Die Anwendung wurde im Rahmen einer IT-Umschulung zur Fachinformatikerin Anwendungsentwicklung am Robotron Bildungszentrum Leipzig entwickelt. Sie richtet sich an Nutzerinnen und Nutzer, die Wetterdaten mehrerer Städte auf einen Blick vergleichen und die Entwicklung über einen längeren Zeitraum verfolgen möchten.

Der vollständige Quellcode ist versionskontrolliert auf GitHub verfügbar. Eine öffentlich erreichbare Demo läuft auf Streamlit Community Cloud unter:
<https://staedtedashboard-4uuuttdjh5rcy9k7ht3bhw.streamlit.app>

2. Architektur

Die Anwendung folgt einer strikten Schichtentrennung nach dem Prinzip der Separation of Concerns:

Frontend → Logikschicht → Datenzugriffsschicht → Datenbank / externe APIs

Jede Schicht kommuniziert ausschließlich mit der direkt benachbarten Schicht. Das Frontend enthält keine Datenbankbefehle und keine direkten API-Aufrufe. Die Datenzugriffsschicht kennt die Oberfläche nicht. Diese Trennung macht einzelne Komponenten austauschbar und den Code testbar.

Modul	Schicht	Aufgabe
-------	---------	---------

app.py	Frontend	Streamlit-Hauptseite — Stadtsuche, Wetteranzeige, Weltkarte, Städtevergleich
pages/Historisches_Wetter.py	Frontend	Unterseite — freie Ortssuche, Zeitraumauswahl, Temperatur- und Niederschlagsdiagramme
pages/Reisewetter.py	Frontend	Unterseite — Reisezielsuche, Klimaübersicht als Jahresdiagramm
api_client.py	Backend	Alle API-Abrufe — Open-Meteo, wttr.in, REST Countries, Geocoding, Archiv, Klima
datenbank.py	Datenzugriff	Alle SQLite-Operationen — Anlegen, Einfügen, Abfragen, Löschen
logik.py	Backend	Berechnungen mit pandas — Durchschnitte, Extremwerte, Icon-Mapping
main.py	Einstiegspunkt	Startet die Streamlit-Anwendung

3. Technologiestack

Komponente	Technologie	Version	Begründung
Programmiersprache	Python	3.x	Breite Bibliotheksunterstützung, kurze Entwicklungszeit, gut geeignet für Datenverarbeitung
Frontend	Streamlit	aktuell	Minimaler Boilerplate, sofort lauffähig im Browser, kein HTML/CSS-Templating erforderlich
Datenbank	SQLite	eingebaut	Keine Serverinstallation nötig, direkt in Python integriert, für Einzelnutzer-Anwendungen ausreichend
HTTP-Anfragen	requests	aktuell	Standard-Bibliothek für REST-API-Aufrufe in Python, robuste Fehlerbehandlung

Datenverarbeitung	pandas	aktuell	Aggregationen, Durchschnitte, Vergleichsberechnungen auf tabellarischen Daten
Diagramme	plotly	aktuell	Interaktive Temperatur- und Niederschlagsdiagramme mit Zoom, Hover und Legende
Kartendarstellung	folium	aktuell	Einfache Erstellung interaktiver Leaflet-Karten aus Python heraus
Karten-Integration	streamlit-folium	aktuell	Einbettung von folium-Karten direkt in Streamlit-Anwendungen
Konfiguration	python-dotenv	aktuell	Sichere Trennung von Code und Konfiguration über .env-Dateien
Deployment	Streamlit Community Cloud	—	Kostenlos, direkte GitHub-Integration, keine Serverkonfiguration erforderlich

4. API-Anbindung

Die Anwendung nutzt sieben externe Dienste. Alle API-Aufrufe sind vollständig mit try/except abgesichert. Bei Timeout oder Verbindungsfehlern erhält die Nutzerin eine verständliche Fehlermeldung statt eines Anwendungsabsturzes. Der Timeout-Wert ist über die .env-Datei konfigurierbar (API_TIMEOUT, Standard: 10 Sekunden).

4.1 Open-Meteo Forecast API

- **URL:** <https://api.open-meteo.com/v1/forecast>
- **Verwendung:** Abruf aktueller Wetterdaten sowie einer 7-Tage-Prognose pro Stadt
- **Authentifizierung:** keine — der Dienst ist vollständig kostenlos und benötigt keinen API-Schlüssel
- **Abgerufene Felder:**
 - Aktuell: current_weather (Temperatur, Windgeschwindigkeit, Wettercode)
 - Prognose: temperature_2m_max, temperature_2m_min, precipitation_sum, weathercode
- **Fehlerbehandlung:** Bei HTTP-Fehlern, Timeout oder unerwarteter Antwortstruktur wird automatisch auf wttr.in umgeschaltet

4.2 Open-Meteo Geocoding API

- **URL:** <https://geocoding-api.open-meteo.com/v1/search>

- **Verwendung:** Automatische Koordinatenermittlung aus einem Stadtnamen. Die Nutzerin muss keine Koordinaten manuell eingeben — der Stadtname genügt
- **Authentifizierung:** keine
- **Besonderheit:** Die API gibt eine gerankte Liste von Treffern zurück. Bei mehrdeutigen Städtenamen (z. B. "Frankfurt" in Deutschland und den USA) wird der Nutzerin eine Auswahl angezeigt. Bei eindeutigem Treffer erfolgt die Speicherung automatisch
- **Abgerufene Felder:** name, country, admin1 (Region), latitude, longitude

4.3 wttr.in (Fallback-API)

- **URL:** <https://wttr.in/{stadtname}?format=j1>
- **Verwendung:** Automatischer Fallback bei Ausfall oder Fehler von Open-Meteo. Der Wechsel erfolgt transparent ohne Nutzereingriff — ein Hinweis in der Oberfläche informiert über den eingeschränkten Betrieb
- **Authentifizierung:** keine
- **Einschränkungen:**
 - Liefert maximal 3 Tage Prognose statt 7
 - Wettercodes weichen vom WMO-Standard ab — Wettericons können im Fallback-Betrieb nicht korrekt zugeordnet werden (Anzeige: N/A)
- **Abgerufene Felder:** temp_C, windspeedKmph, precipMM, weatherCode, mintempC, maxtempC

4.4 REST Countries

- **URL:** <https://restcountries.com/v3.1/name/{landname}>
- **Verwendung:** Abruf von Länderinformationen für den Karten-Popup jeder Stadt
- **Authentifizierung:** keine
- **Abgerufene Felder:** capital (Hauptstadt), currencies (Währung mit Symbol), languages (Landessprachen)
- **Fehlerbehandlung:** Bei Fehler wird der Popup ohne Länderinformationen angezeigt — die Kernfunktionalität bleibt unbeeinträchtigt

4.5 OpenStreetMap / CartoDB Positron (Kartendienst)

- **Bibliotheken:** folium, streamlit-folium
- **Kartendaten:** OpenStreetMap-Tiles über CartoDB Positron — ein heller, reduzierter Kartenstil, der die Stadtpins gut zur Geltung bringt

- **Verwendung:** Interaktive Weltkarte mit einem Pin pro gespeicherter Stadt. Ein Klick auf einen Pin öffnet einen Popup mit aktuellen Wetterdaten (Temperatur, Wind, Niederschlag) sowie Länderinformationen aus REST Countries (Hauptstadt, Währung, Sprache)
- **Authentifizierung:** keine — OpenStreetMap-Tiles sind frei nutzbar
- **Technischer Ablauf:** Beim Aufbau der Karte wird für jeden Stadtpin ein separater API-Aufruf an Open-Meteo bzw. wtrr.in ausgeführt, um aktuelle Wetterdaten in den Popup zu laden. Schlägt ein Abruf fehl, wird der betreffende Pin ohne Wetterdaten übersprungen
- **Einbettung:** Die fertige folium-Karte wird über streamlit-folium direkt in die Streamlit-Oberfläche eingebettet und ist vollständig interaktiv (Zoom, Pan, Klick)

4.6 Open-Meteo Archive API

- URL: <https://archive-api.open-meteo.com/v1/archive>
- Verwendung: Abruf historischer Wetterdaten für beliebige Orte ab 1940
- Authentifizierung: keine
- Abgerufene Felder: temperature_2m_max, temperature_2m_min, precipitation_sum
- Darstellung: Liniendiagramm Temperaturverlauf, Balkendiagramm Niederschlag

4.7 Open-Meteo Climate API

- URL: <https://climate-api.open-meteo.com/v1/climate>
- Verwendung: Klimatologische Monatsdurchschnitte für Reiseziele, Zeitraum 1991–2020
- Authentifizierung: keine
- Modell: EC_Earth3P_HR
- Darstellung: Jahresübersicht mit Temperatur- und Niederschlagsdiagramm

5. Datenbankmodell

Die lokale SQLite-Datenbank liegt unter daten/projekt.db und wird beim ersten Anwendungsstart automatisch angelegt (CREATE TABLE IF NOT EXISTS). Sie enthält drei Tabellen:

staedte : Verwaltung der gespeicherten Städte

Feld	Typ	Beschreibung
id	INTEGER PRIMARY KEY AUTOINCREMENT	Eindeutiger Schlüssel

name	TEXT NOT NULL UNIQUE	Stadtname
land	TEXT	Landesname
breitengrad	REAL	Geografische Breite (Dezimalgrad)
laengengrad	REAL	Geografische Länge (Dezimalgrad)
angelegt_am	TEXT	Zeitstempel der Anlage (ISO 8601)

Wetterdaten: Verlauf der abgerufenen aktuellen Wetterdaten

Feld	Typ	Beschreibung
id	INTEGER PRIMARY KEY AUTOINCREMENT	Eindeutiger Schlüssel
stadt_id	INTEGER	Fremdschlüssel → staedte.id
abgerufen_am	TEXT	Zeitstempel des Abrufs (ISO 8601)
temperatur	REAL	Aktuelle Temperatur in °C
windgeschwindigkeit	REAL	Windgeschwindigkeit in km/h
niederschlag	REAL	Niederschlag in mm
wettercode	INTEGER	WMO-Wettercode

Prognose: Gespeicherte 7-Tage-Prognosedaten

Feld	Typ	Beschreibung
id	INTEGER PRIMARY KEY AUTOINCREMENT	Eindeutiger Schlüssel
stadt_id	INTEGER	Fremdschlüssel → staedte.id
datum	TEXT	Prognosedatum (YYYY-MM-DD)
temperatur_min	REAL	Tagestiefsttemperatur in °C
temperatur_max	REAL	Tageshöchsttemperatur in °C
niederschlag	REAL	Niederschlagssumme in mm
wettercode	INTEGER	WMO-Wettercode

Wetterdaten werden bei jedem Seitenaufruf neu abgerufen und gespeichert. Dadurch baut sich über Zeit ein Verlauf auf, aus dem Durchschnittswerte und Extremwerte mit pandas berechnet werden.

6. Designentscheidungen

Streamlit statt Flask

Für ein erstes Python-Projekt mit Fokus auf Backend-Logik und Datenverarbeitung bietet Streamlit den schnellsten Weg zu einer lauffähigen, ansehnlichen Webanwendung. Flask hätte eigene HTML-Templates, Routing und Session-Management erfordert und damit den Entwicklungsaufwand deutlich erhöht, ohne einen funktionalen Mehrwert zu liefern.

Streamlit Pages für Unterseiten

Die Unterseiten Historisches Wetter und Reisewetter werden über den pages/Mechanismus von Streamlit realisiert. Jede .py-Datei im pages/-Ordner erscheint automatisch in der Navigation — ohne eigenes Routing.

SQLite statt PostgreSQL

Für eine Einzelnutzer-Anwendung ohne Mehrbenutzer-Anforderungen ist SQLite ausreichend, wartungsärmer und ohne Serverinstallation nutzbar. Die Datenbank liegt als einzelne Datei im Projektordner und ist damit portabel. Bei zukünftiger Skalierung wäre eine Migration zu PostgreSQL möglich, da der Datenzugriff vollständig in datenbank.py gekapselt ist.

Plotly für Diagramme

Für die historischen und klimatologischen Daten wurde plotly gewählt, da es interaktive Diagramme mit Zoom, Hover-Werten und Legende ohne zusätzliche Konfiguration liefert. Streamlit hat st.plotly_chart() direkt eingebaut.

Open-Meteo als primäre API

Kostenlos, kein API-Schlüssel erforderlich, stabile Verfügbarkeit und eine Geocoding-API im selben Ökosystem. Die automatische Koordinatenermittlung aus Stadtnamen ist ein wesentliches Komfortmerkmal der Anwendung.

wtr.in als Fallback-API

Um die Verfügbarkeit der Anwendung bei Serverausfällen zu gewährleisten, wurde wtr.in als automatischer Fallback implementiert. Der Wechsel erfolgt transparent — die Anwendung zeigt lediglich einen Hinweis auf den eingeschränkten Betrieb. Dies erhöht die Resilienz ohne zusätzliche Konfiguration durch die Nutzerin.

Modulare Struktur

Fünf klar getrennte Module nach Verantwortlichkeit. Änderungen an der API betreffen ausschließlich `api_client.py`, Änderungen am Frontend ausschließlich `app.py`. Diese Trennung erleichtert spätere Erweiterungen und reduziert das Fehlerrisiko bei Anpassungen.

Deployment auf Streamlit Community Cloud

Kostenlos, direkte GitHub-Integration, öffentlich erreichbar unter einer festen URL, kein eigener Server erforderlich. Jeder git push auf den main-Branch löst automatisch ein Redeployment aus.

7. Bekannte Einschränkungen

- Bei Ausfall von Open-Meteo liefert der Fallback `wtr.in` nur 3 statt 7 Tage Prognose
- Wettercodes von `wtr.in` weichen vom WMO-Standard ab — Wettericons werden im Fallback-Betrieb nicht korrekt dargestellt (N/A)
- Wetterdaten werden nicht automatisch aktualisiert — der Abruf erfolgt durch manuelles Neuladen der Seite
- Die Anwendung unterstützt keine Nutzerverwaltung und keinen Mehrbenutzer-Betrieb
- Historische Wetterdaten werden nicht rückwirkend aus der API geladen — der Verlauf beginnt ab dem ersten eigenen Abruf
- Das Hintergrundbild muss als Datei im Repository liegen und kann nicht per externer URL eingebunden werden, da Streamlit externe CSS-Hintergrundbilder blockiert
- Auf Streamlit Community Cloud wird die SQLite-Datenbank bei jedem Redeployment zurückgesetzt — gespeicherte Wetterdaten gehen verloren

8. Potenziale

- **Wetterhistorie als Diagramm:** Die Daten werden bereits gespeichert — eine Visualisierung als Liniendiagramm mit `plotly` oder `matplotlib` wäre der nächste sinnvolle Schritt
- **Wettercode-Mapping für `wtr.in`:** Korrekte Icon-Darstellung auch im Fallback-Betrieb durch Mapping der `wtr.in`-Codes auf WMO-Codes
- **Automatische Aktualisierung:** Hintergrundprozess mit `schedule` oder einem Streamlit-Timer für regelmäßige Datenaktualisierung ohne manuelles Neuladen

- **REST Countries vollständig integrieren:** Länderinformationen könnten auch direkt in den Stadtkacheln erscheinen, nicht nur im Karten-Popup
- **Exportfunktion:** CSV-Download der gespeicherten Wetterdaten für externe Auswertungen
- **Persistente Datenbank auf Streamlit Cloud:** Anbindung an eine externe Datenbank (z. B. PostgreSQL auf Render) für dauerhaften Datenverlauf auch nach Redeployments
- **Interaktive Karte erweitern:** Farbkodierung der Pins nach aktuellem Wettercode oder Temperatur für schnellere visuelle Orientierung
- **Stadtvergleich auf Historisches Wetter:** Mehrere Orte im gleichen Zeitraum vergleichen